Simulating the Deutsch-Jozsa Algorithm Classically

Andy Norman and Daniel Mejenny

Abstract

The Deutsch-Jozsa algorithm has long been used as an example of what makes quantum computers superior to classical ones. In our paper, we simulate the algorithm in a classical machine using Qiskit software. We also present and summarize scholarly literature on quantum and classical simulations of the Deutsch-Jozsa algorithm using QSL and probabilistic bits. Finally, considering our simulation and the results of our research, we posit that the Deutsch-Jozsa problem is not the best example of quantum superiority.

Introduction

In the realm of quantum computing, the Deutsch-Jozsa Algorithm is commonly used as an undeniable example of quantum superiority. It is a fairly simple algorithm which uses the quantum qualities of parallelism and interference to show a significant, exponential speed-up compared to classical computing. In the Deutsch-Jozsa problem, you are given a function of length n binary bits and are promised it is either constant (made up entirely of 0 or 1 bits) or balanced (made up of half 0 and half 1 bits). The function is embedded in a black-box oracle which the user can query to determine if it is constant or balanced. A quantum circuit can solve

the problem by applying a Hadamard gate to n+1 input qubits, putting those super-positioned qubits into the oracle, applying a Hadamard gate to the first n qubits once they've come out of the oracle, and measuring the resulting output sequence. If phase kickback is observed, you know the function is balanced, and if it's not observed, you know the function is constant. While a quantum circuit needs just 1 query regardless of n, a classical circuit requires $(2^n/2)+1$ queries to solve the problem.

In recent years, efficient simulations of the quantum solution to the Deutsch-Jozsa Algorithm on a classical computer have been discovered. In 2017, quantum researcher Niklas Johansson showcased his simulation of the algorithm using Quantum Simulation Logic (QSL) on a classical deterministic machine, which was able to obtain the same success probability and the same time and space complexity as for the quantum algorithm. Just last year in 2023, the Deutsch-Jozsa algorithm was shown to also be simulated effectively and efficiently using classical probabilistic circuits and bits.

Our goal with our research is twofold. First, we want to make our own simulation of the Deutsch-Jozsa algorithm using Qiskit to reinforce our understanding of it, especially in cases where n>2. Second, given the recent discovery of classical simulations of the algorithm, we want to revisit and possibly reconsider the notion that the Deutsch-Jozsa algorithm undeniably shows quantum speed up.

Methods

Before attempting to simulate the algorithm ourselves, we searched for literature discussing existing classical simulations of the Deutsch-Jozsa algorithm. This consisted of entering key search terms into the Loyola Notre Dame Library “OneSearch” tool and into Google Scholar. We quickly found the article by Deniz Yavuz and Anirudh Yadav about simulating the Deutsch-Jozsa algorithm with a classical probabilistic method. This source was used in tandem with Niklas Johansson’s article describing his QSL simulation of the Deutsch-Jozsa algorithm, which was given to us at the assignment of the research project.

Then, using assistance from the Qiskit algorithms tutorial page, we were able to create a simulation of the Deutsch-Jozsa algorithm. In our simulation, we first created a code that when given an nth number of registers, can at random choose between a constant or balanced function.

```
n = 5 # the length of the first register for querying the oracle \

# Choose a type of oracle at random. With probability half it is constant,
# and with the same probability it is balanced
oracleType, oracleValue = np.random.randint(2), np.random.randint(2)

if oracleType == 0:
    print("The oracle returns a constant value ", oracleValue)
else:
    print("The oracle returns a balanced function")
    a = np.random.randint(1,2**n) # this is a hidden parameter for balanced oracle.
```

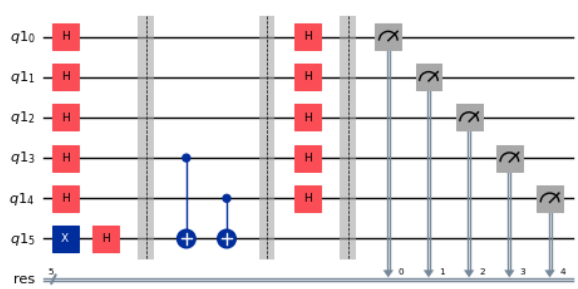
✓ 0.0s

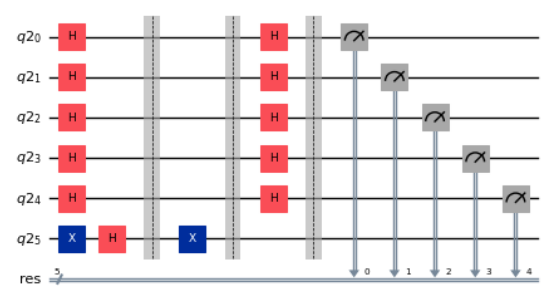
```
The oracle returns a balanced function
```

```
n = 5 # the length of the first register for querying the oracle \

# Choose a type of oracle at random. With probability half it is constant,
# and with the same probability it is balanced
oracleType, oracleValue = np.random.randint(2), np.random.randint(2)

if oracleType == 0:
    print("The oracle returns a constant value ", oracleValue)
else:
    print("The oracle returns a balanced function")
    a = np.random.randint(1,2**n) # this is a hidden parameter for balanced oracle.
```

✓ 0.0s

```
The oracle returns a constant value  1
```



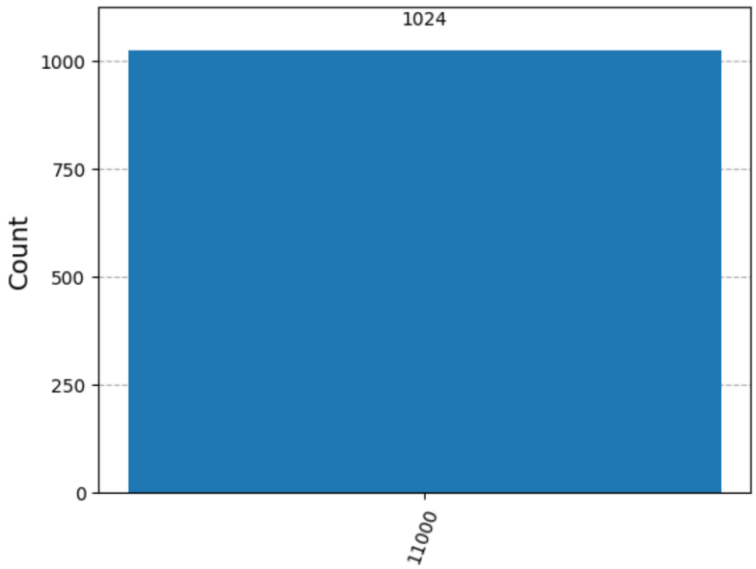


Given this function our code was able to generate

a corresponding circuit including Pauli-X gates, CNOT gates, and Hadamard gates. Finally, the qubit output can be measured in a histogram shown by:

Results

Our code was able to effectively simulate the Deutsch-Jozsa algorithm for values of n<20.



However, an important distinction must be made between our Qiskit simulation and the true quantum algorithm. In a quantum circuit, phase kick-back is observed after the input qubits leave the black-box oracle and the Hadamard gates are applied if the function is balanced. In our Qiskit simulation, for a balanced case, the black-box oracle is replaced by visible CNOT gates which are what enable the phase kick-back (for a constant case, the CNOT gates are absent). In the simplest terms, our simulation uses a “transparent” oracle instead of a black-box oracle. So, while we can determine that the Deutsch-Jozsa algorithm can be simulated effectively in a classical computer, we must concede that it is not entirely accurate portrayal of the quantum solution due to this discrepancy.

Discussion

Though our simulation was not a true representation of the Deutsch-Jozsa algorithm, it succeeded in deepening our understanding of the algorithm and of quantum computing in general. Ironically, relying on a transparent box proved to be beneficial because it helped us understand what the oracle did and contained in the context of the Deutsch-Jozsa algorithm. Being able to "see through" the oracle and observe the CNOT gates aided in our understanding of phase kick-back—seeing our code simulate the algorithm multiple times made it clearer that phase kick-back being observed is the deciding factor between balanced vs constant. Similarly, being able to see the algorithm simulated for cases where n>2 was also helpful. As n increased linearly, the number of input qubits and Hadamard gates also increased linearly, and the circuit was still able to solve the algorithm with just one query. Even though that was the result we were expecting given our previous understanding of the Deutsch-Jozsa algorithm, seeing it simulated in real time was certainly a helpful experience.

Johansson's method of simulating the Deutsch-Jozsa algorithm classically using QSL resulted in an error probability of zero but had the same caveat as our simulation: it relies on a "transparent" oracle rather than the traditional black box. Like our Qiskit simulation, his QSL circuit's oracle is constructed of visible CNOT gates in the case of a balanced circuit. Despite this distinction, Johansson still concludes that because the Deutsch-Jozsa can be simulated effectively in a classical machine, it simply does not require a genuine quantum circuit or computer [1].

Yavuz and Yadav's probabilistic simulation involved representing qubits as vectors in an eight-dimensional probability space (deemed "simplex" vectors) and representing quantum gates with specific transformative operations on these simplex vectors. Using these baselines, they were able to perform a classical simplex simulation of the Deutsch-Jozsa algorithm, achieving an

identical result to the quantum solution. Notably, their oracle was much more comparable to the quantum black-box than the transparent oracle in our or Johansson's simulation in the way it embedded the function—phase kick back could only be observed in the output; there was no way to see into the oracle and determine constant or balanced simply by the oracle's make up. In other words, the probabilistic simulation created a genuine black-box oracle. Though more research on the noise and error correction of classical probabilistic circuits simulating complex quantum algorithms is required, in the case of the Deutsch-Jozsa algorithm, the probabilistic computation proved to be successful [2].

Conclusion

We successfully simulated the Deutsch-Jozsa algorithm using Qiskit. While our simulation serves as an excellent visual representation of the algorithm and built upon our understanding of its solution, it is not a true representation of the algorithm because of the transparency of the oracle. However, we were able to research and find a classical simulation of the Deutsch-Jozsa algorithm with a genuine black-box oracle using probabilistic bits. The success of our simulation in Qiskit in conjunction with the general simulatability of the algorithm using various classical methods leads us to conclude that the Deutsch-Jozsa algorithm does not showcase quantum superiority as much as it once was perceived to have done.

References

[1] Johansson, N., Larsson, JÅ. Efficient classical simulation of the Deutsch–Jozsa and Simon's algorithms. *Quantum Inf Process* **16**, 233 (2017) DOI https://doi.org/10.1007/s11128-017-1679-7

[2] Yavuz, D. D., and A. Yadav. "Simulation of quantum algorithms using classical probabilistic bits and circuits." *arXiv preprint arXiv:2307.14452* (2023) DOI https://doi.org/10.48550/arXiv.2307.14452